CSC 540: Project 1 (Database Application Development Project)

Freight Tracking and Delivery Management for a Regional Logistics Carrier

Due Dates

- **Deliverable 1:** Preliminary Report (ER model + relational schema), may be partial (**Sep 28th, 11:59pm**)
- **Deliverable 2:** Full implementation (**Nov 14th, 11:59 pm**)
- **Deliverable 3:** Demos (**TBA-around the 3rd week of Nov**)

Target DBMS: MariaDB 10.5 (the departmental class server, classdb2.csc.ncsu.edu). Note that although the service is referred to as "MySQL," the server is MariaDB; develop and test against MariaDB behavior. Access is restricted to campus IP addresses (use the campus VPN off campus). Instructions can be found at https://it.csc.ncsu.edu/mysql/. Log in with UnityID as username. The initial password is set as studentid.

Client: A menu-driven interface in JDBC (Java) or Python is sufficient. **No ORM tools are allowed as part of your development stack.**

Link of Application Flow: **Project Application Flow**

Link of Sample Data: **Sample Data for Demo and Testing.docx**

Peer review Link: TBA. A link will be provided to submit your review of your assessment of your team members' contribution to the project.

Application Domain Overview

This project centers on a regional freight carrier that moves shipments through a network of hubs. The company operates across a few regions, works with a handful of partner carriers, and runs scheduled departures between hubs. Its customers hand off freight to be delivered by a promised date. The database is the operational backbone: it records what is being shipped, who is moving it, where each shipment currently is, and whether deliveries are meeting their commitments.

Hubs, regions, and lanes

A hub is a physical facility (a depot, terminal, or sorting center). Every hub belongs to exactly one region. A region is a named geographic grouping of hubs, and for this project, the set of regions is fixed: Northeast, Southeast, Midwest, Southwest, and West. A region is simply a property of a hub (a hub's region), not an entity with data of its own. Freight moves between hubs. An ordered pair of hubs (origin, destination) is called a lane. Lanes are the natural unit for analysis: management asks questions like which lanes carry the most volume, or which lanes miss their delivery targets most often. The network is moderate in size: five regions, a few dozen hubs, and multiple lanes between them.

Carriers, vehicles, and departures

Freight is physically moved by carriers. A carrier owns vehicles, and a vehicle runs a departure: a specific vehicle leaving a specific origin hub at a specific date and time, bound for a destination hub. Every

departure has a finite capacity, expressed as a maximum total weight in kilograms. Capacity is the scarce resource in this domain: shipments are booked onto a departure only while their combined weight stays within that maximum. A shipment whose weight alone exceeds a departure's maximum cannot be booked onto that departure.

Vehicles come in different types, and the type matters because different types are described by different attributes. The fleet includes, for example, box trucks (with a cargo volume and a gross weight rating), refrigerated vans (with a temperature range and a smaller payload limit), and rail cars (with an axle count and a track gauge). All vehicles share some common attributes (an identifier, the owning carrier, a home hub), but each type also carries attributes that make no sense for the others. A refrigerated van has a temperature range; a railcar does not. Your design should account for both what vehicles have in common and what is specific to each type.

Shipments

A shipment is a customer's freight moving from an origin to a destination, with a weight (in kilograms) and a promised delivery date (its service commitment, or SLA; see Delivery commitments below). A shipment is created by a shipper, then booked onto a departure that will carry it. As the shipment travels, it accumulates a history of scan events.

Scan events (the movement history)

Each time a shipment is handled, a scan event is recorded: a timestamp, the hub where it was scanned, and a status (for example: received, in transit, arrived at hub, out for delivery, delivered). A scan event has no meaning on its own. It exists only as part of a particular shipment's journey, and it is identified by its position within that shipment's sequence of scans rather than by any identifier that would make sense in isolation. Two different shipments may each have a "first scan" or a scan at the same hub at the same minute; what makes a scan unique is the shipment it belongs to, together with its place in that shipment's ordered history. If a shipment record were removed, its scans would have no reason to exist. The ordered sequence of scans for a shipment is its movement history. This history is the heart of the database. It supports tracking (where is my shipment now?), performance analysis (how long did it sit at each hub?), and delivery verification (was it delivered on time?). Scans for a shipment are chronologically ordered, and a shipment cannot reach a later status before an earlier one (it cannot be delivered before it is in transit).

Booking onto a departure (the capacity rule)

Booking a shipment onto a departure must respect capacity. A booking succeeds only if the shipment's weight, added to the weight already booked on that departure, would not exceed the departure's maximum weight; otherwise, it is rejected. Because several shipments may compete for the last remaining kilograms on a departure, booking is performed as a single atomic operation: the check of available weight and the recording of the booking happen together, so a departure can never be overbooked. This is implemented as a stored procedure (see Required Procedures and Triggers).

Delivery commitments (SLAs)

Every shipment carries a promised delivery date, set when the shipment is created. This date is the shipment's service commitment (SLA). It is stored explicitly on the shipment; it is not derived from any

other date. On-time performance is defined precisely, so that reporting is unambiguous: a shipment is on time if the timestamp of its delivering scan (the scan whose status is delivered) falls on or before the end of its promised delivery date, and late if that scan falls after the end of the promised delivery date. A shipment that has no delivery scan yet is neither on time nor late; it is still in transit and is excluded from on-time and late counts. On-time performance is a central reporting concern: the company tracks on-time rates by carrier, by lane, and over time.

Analytics over the movement history

Beyond day-to-day operations, the company runs analytical queries over its shipments and scan history. These include rankings (the busiest lanes, the top departures by load), running totals (cumulative volume over a period), sequential analysis (the time a shipment spends between consecutive scans), and on-time trends. Several of these are most naturally expressed with SQL window functions, which are a required part of this project (see Queries to be Implemented).

Access control

Each account acts within its own scope. A shipper acts only on the shipments it owns. A carrier acts only on the departures and vehicles it owns (it can post scans only for departures it operates). A viewer has read-only access. These rules are enforced at the application level using role selection and passwords; no database-level privilege management is required.

Functional Requirements

Roles

The first screen lets the user select a role. Implement exactly these three roles. A user holds exactly one role.

- **Shipper**: creates and manages shipments, sets each shipment's promised delivery date, books shipments onto departures, and tracks and reports on its own shipments.
- **Carrier**: manages its vehicles and scheduled departures (including capacity), posts scan events as freight moves, and runs operational reports.
- **Viewer**: read-only access to shipment tracking and published performance reports.

Business rule: for this project, a user holds exactly one role. No database-level privileges are required; use application-level role management with passwords.

Shipper: Functions

Create and Manage Shipments

Menu: Shipper → Shipments → Create/Update.

Create a shipment with an origin hub, destination hub, weight, and a promised delivery date. Each shipment is owned by the current shipper account. The (origin, destination) pair determines the shipment's lane.

Book a Shipment for a Departure

Menu: Shipper → Shipments → Book.

Select one of the shipper's shipments and an eligible departure (one whose origin and destination match the shipment's origin and destination, and that still has room for the shipment's weight). Booking is performed atomically and succeeds only if the shipment's weight fits within the departure's remaining capacity; otherwise, it is rejected with a clear message. On success, the departure's booked weight increases by the shipment's weight, and the booking is recorded.

Track a Shipment

Menu: Shipper → Shipments → Track.

For one of the shipper's shipments, show its full movement history (all scan events in chronological order) and its current status.

Shipper Reports

Menu: Shipper → Reports.

On-time vs. late summary for the shipper's own shipments, and a list of the shipper's shipments currently in transit.

Carrier: Functions

Manage Vehicles

Menu: Carrier → Vehicles → Create/Update.

Maintain the set of vehicles this carrier owns.

Manage Departures

Menu: Carrier → Departures → Create/Update.

Schedule a departure: a vehicle, an origin hub, a destination hub, a departure date and time, and a maximum total weight in kilograms. A carrier may schedule departures only for vehicles it owns.

Post a Scan Event

Menu: Carrier → Scans → Post.

Record a scan for a shipment the carrier is handling: timestamp, hub, and new status. The system maintains the shipment's current status and enforces chronological, non-regressing status order. A carrier may post scans only for departures it operates.

Carrier Reports

Menu: Carrier → Reports.

Weight utilization per departure (booked weight against maximum weight, as an amount and a percentage), and the carrier's on-time delivery rate. See also the analytical queries in the next section.

Viewer: Functions

Track a Shipment

Menu: Viewer → Track.

Given a shipment identifier, show its movement history and current status (read-only).

Published Performance Reports

Menu: Viewer → Reports.

View published, read-only performance summaries (for example, on-time rates by lane or by carrier).

Access Control (applies to all): shippers act only on shipments they own; carriers act only on vehicles and departures they own, and post scans only for departures they operate; viewers are read-only.

View Queries (available from the root menu, all roles)

In addition to the role-specific functions above, the root menu (the first menu, before or alongside role selection) must include a View Queries option. Selecting it displays a numbered list of the analytical queries from the Queries to be Implemented section, each accompanied by a short text description. Selecting a number runs that query and displays its results. Where a query takes a parameter (for example, a shipment, a carrier, a period, or a threshold N), prompt for it, then run the query and show the output.

This is a required feature. Expose every query from the Queries to be Implemented list here, numbered in the same order as that list, so the queries can be located and run consistently. This is the single place where all of the required analytical queries are demonstrated.

Queries to be Implemented

Final list of queries to be implemented. Include a screen in your application that displays these numbers as mentioned in the View Queries Section.

1. Show the number of vehicles of each type owned by each carrier.
2. For each lane, rank departures by weight utilization (booked weight as a percentage of maximum weight), and list the most heavily loaded departures per lane.
3. For a given shipment, compute the time elapsed between each consecutive pair of scan events (the dwell time at each step of its journey).
4. For each carrier, compute a running total of shipment weight delivered over time (for example, cumulative delivered kilograms by week).
5. Find departures carrying shipments belonging to at least two different shippers. Show the departure ID and the number of distinct shippers.
6. List shipments that have no scan events recorded. Include the shipment ID, owning shipper, origin hub, destination hub, and promised delivery date.

Required Procedures and Triggers

Keep these focused. One procedure and two triggers are required. Each must do real work that is best handled inside the database.

Procedure: Book Shipment onto Departure

Given a shipment and a departure, atomically: compute the weight already booked on the departure (the sum of the weights of shipments already booked onto it), verify that adding this shipment's weight would not exceed the departure's maximum weight, and if it fits, record the booking. If the shipment's weight would push the total over the maximum, make no change and report failure. Because multiple bookings may compete for the last available kilograms, the summation, the check, and the insert must occur as one unit (a single transaction with appropriate locking) so a departure can never be overbooked. Note that this requires an aggregate over existing bookings inside the transaction, not merely reading a counter.

Trigger: Maintain Status and Flag Late Delivery

On the insertion of a scan event, update the shipment's current status to the scan's status. If the scan records delivery, determine whether the shipment met its commitment using the on-time rule defined above (delivered on or before the end of the promised delivery date) and record the on-time or late outcome on the shipment. This keeps the derived delivery state correct without relying on application code.

Trigger: Enforce Scan Chronology

On the insertion of a scan event, reject the scan if it would violate chronological order or cause the shipment's status to regress (for example, a delivered shipment cannot later be scanned as in transit, and a scan cannot predate the shipment's most recent scan). This protects the integrity of the movement history.

> *You may add a supporting view or two if it helps (for example, a view of each departure's booked weight and remaining capacity, or of each shipment's current status and on-time outcome). Views are optional.*

Non-Functional / Implementation Notes

- Use only MariaDB 10.5 features; keep to portable, standard SQL where possible.
- Implement the stored procedure(s) and trigger(s) described below. Assess your design partly on how well you use the DBMS to enforce rules, rather than pushing that logic into application code.
- The application UI may be a menu-driven CLI. The focus is on database modeling and logic, not interface design.

Final Deliverable

Each team will need to book a time slot to demo the application to teaching assistants. Details about the demo will be shared later, but it will be the week of November 9th. The following needs to be submitted: create a zip file and use the file-naming convention from the previous submission.

1. An updated ER model. A short paragraph about what improvements were made from the first design.
2. Three SQL files: The first one should contain SQL for triggers, tables, constraints, and procedures. The second file should contain queries to populate the tables with sample data. The third file should contain the SQL queries listed in the Queries to be Implemented section.
3. Executable file and source Java /Python Code
4. README.txt - This should contain the names of the members and any additional instructions that you think will be necessary to compile and execute your code independently by TAs.
5. A report that discusses the following items. The main report need not be very long (no longer than 4 pages) and may not include figures, tables, etc., which you can put in an appendix.
    a) Functional Dependencies: Functional dependencies that influenced your design, e.g., any that

led to relational decompositions

b) Comments about normalization (which tables are in BCNF and which are in 3NF). None should be below these. (You can use a table representation for this.

c) Constraints: A description of constraints that were not implemented as part of table definitions, and why and how they were implemented in the final design. In particular, a separate subsection here should highlight constraints that couldn't be implemented in the database at all and had to be implemented in application code. Note that a key part of assessing your design is how well you used the DBMS to implement constraints v/s implementing in application code.

Grading

Deliverable	Percentage
Original ER Diagram and Report	10
Revised ER Diagram	15
Report including discussion on constraints, FDs, normalization choices, SQL files + Readme, etc	35
Application Demo Test Cases	40
Peer Review	Average of peer review grades (individual grade is a function of team grade and peer review)